

Développement d'application avec IHM

Gestion des événements



Christine Jacquin

La programmation événementielle

- Le développement d'applications avec IHM est basée sur un paradigme de programmation nommé **programmation événementielle**

- **Programmation impérative (séquence d'instructions)**

le programme est le chef d'orchestre (par exemple, il demande à l'utilisateur d'entrer des valeurs, calcule et affiche un résultat, etc.

- **Programmation événementielle**

- ce sont les événements (généralement déclenchés par l'utilisateur, mais aussi par le système) qui pilotent l'application.

- la programmation événementielle nécessite qu'un processus (en tâche de fond) surveille constamment les actions de l'utilisateur susceptibles de déclencher des événements qui pourront être ensuite traités (ou non) par l'application

Les événements

Un événement peut être provoqué par :

Une action de l'utilisateur

- Un clic avec la souris
- L'appui sur une touche du clavier
- Le déplacement d'une fenêtre
- Un geste sur un écran tactile

...

Un changement provoqué par le système

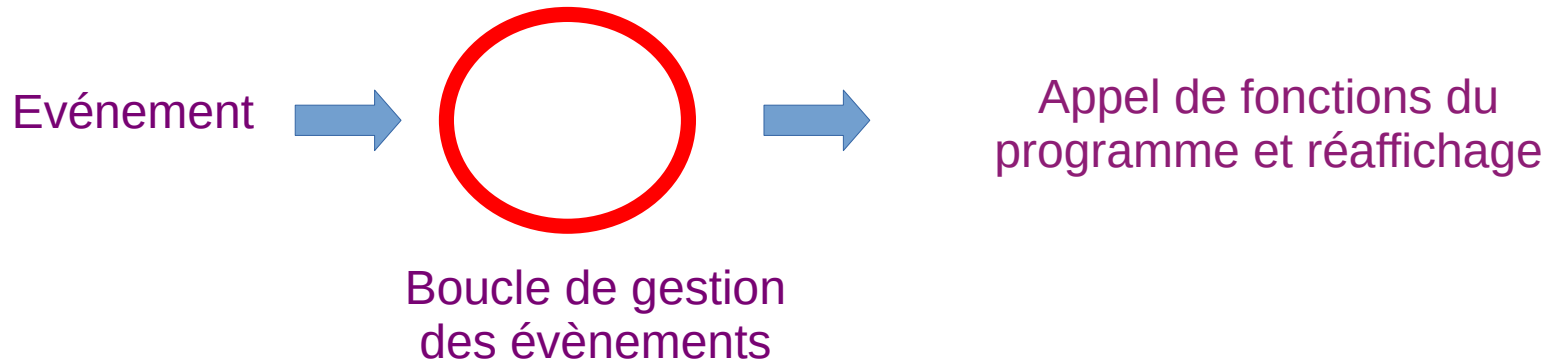
- Un timer est arrivé à échéance
- Un processus a terminé un calcul
- Une information est arrivée par le réseau

...

La boucle de gestion des événements

Boucle infinie qui :

- Récupère les événements
- Notifie les composants
- Lancée automatiquement à l'initialisation du programme



Qu'est-ce qu'un événement en javaFX ?

Un objet de la classe **Event** à partir duquel on peut connaître :

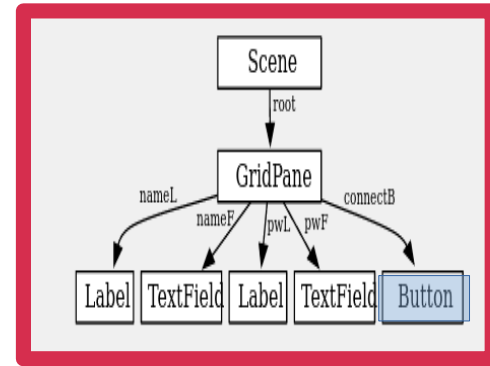
- **Le type** : par exemple, la classe **KeyEvent** qui correspond aux événements liés au clavier qui englobe *KEY_PRESSED*, *KEY_RELEASE*, *KEY_TYPED*. On y accède via l'attribut ***eventType***
- **La source** de l'événement => clavier, souris via l'attribut ***source***
- **La cible** d'un événement => élément (Node) sur lequel l'événement s'est produit (un bouton par exemple). On y accède via l'attribut ***target***

Le " stage " JavaFX reçoit les événements et les diffuse au composant cible

Préalable à la gestion des événements

- sélection par l'utilisateur de la cible (Target) de l'événement

Si plusieurs composants se trouvent à un emplacement donné, celui qui est "au-dessus" est considéré comme la cible



- détermination de la chaîne de traitement des événements

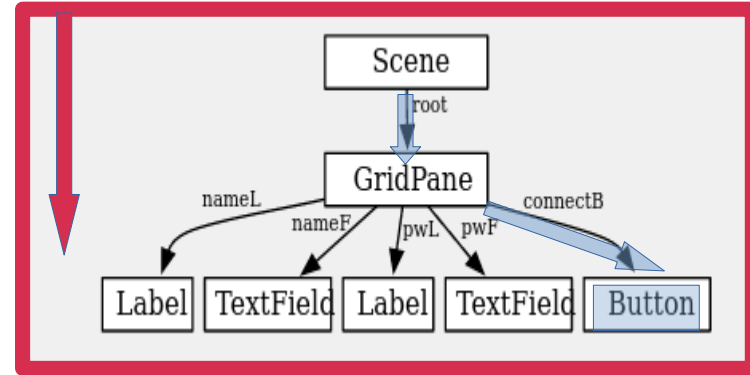
Le chemin part de la racine (*Stage*) et va jusqu'au composant cible en parcourant tous les nœuds intermédiaires. L'objet *stage* propage l'événement sur le chemin construit.

Phase de capture des événements

Lorsque le bouton est cliqué, la chaîne de traitement est générée par l'objet *Stage* et l'évènement est propagé dans l'arbre des nœuds jusqu'au nœud cible (*Button* ici)

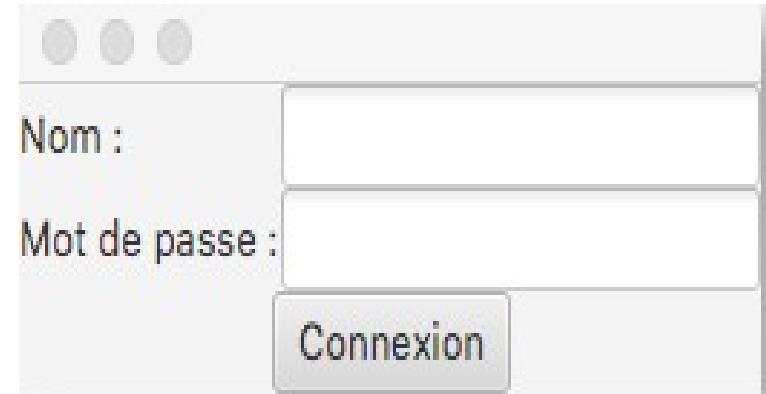
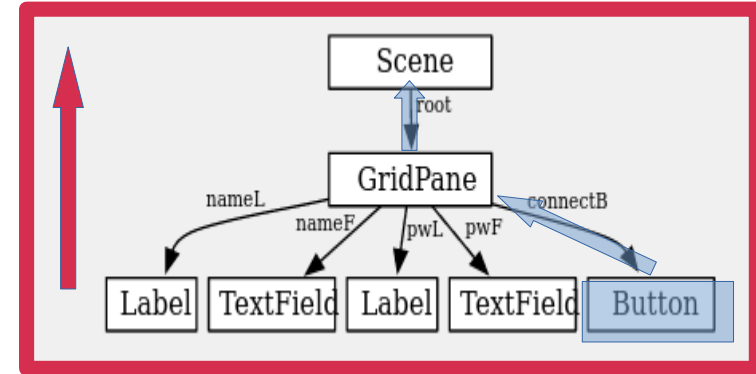
Si un nœud traversé comporte un **filtre d'évènement** alors il est exécuté (dans l'ordre de passage). L'évènement se propage jusqu'au nœud cible

Il est possible au niveau de cette phase d'arrêter la propagation, en utilisant la méthode *consume()* dans le filtre.

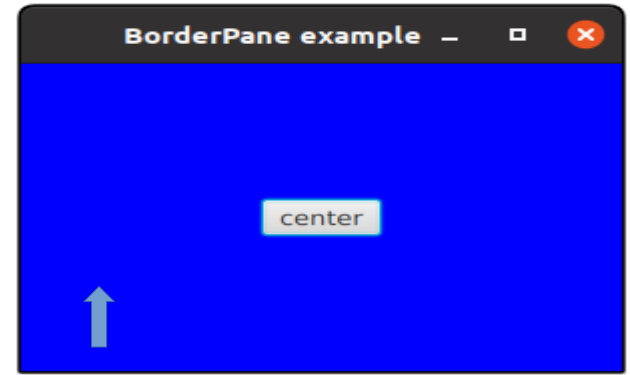
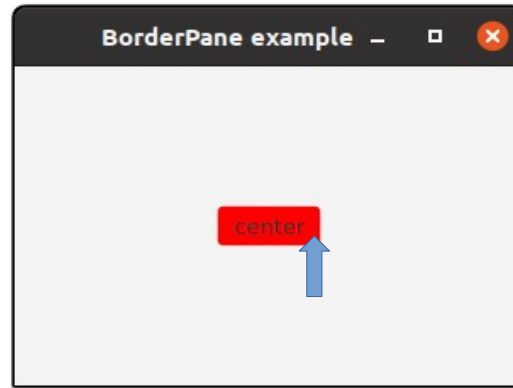


Phase de remontée de l'événement

L'événement remonte ensuite depuis la cible (*Button*) jusqu'à la racine (*Stage*). Lorsqu'un nœud dans le chemin est rencontré possédant un **gestionnaire d'événement** éligible le code de ce dernier est exécuté. La propagation s'arrête alors.



Exemple de traitement d'événement (1)

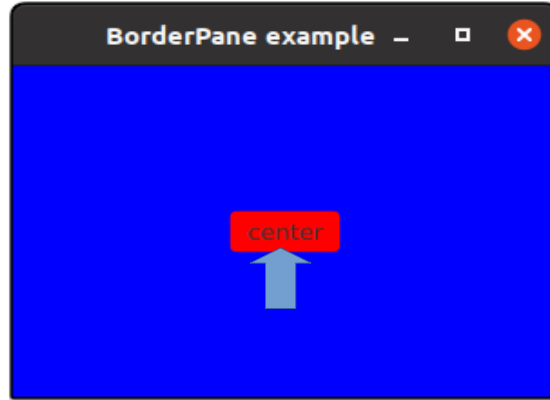


Des **gestionnaires d'événement** sont associés au *Button* et au *BorderPane* :

Button => au clic, il se colore en rouge

BorderPane => au clic, il se colore en bleu

Exemple de traitement d'événement (2)



Maintenant, **un filtre d'événement** est associé au *BorderPane* qui au clic, colorie le panneau en bleu. Le bouton a toujours le même gestionnaire d'événement qu'à la diapo précédente.

Par le mécanisme de descente et de remontée, les 2 écouteurs sont déclenchés au clic sur le bouton

La gestion des événements

Pour traiter un événement, il faut créer un **écouteur** d'évènement (*Event Listener*) et l'enregistrer sur les nœuds du graphe de scène où l'on souhaite intercepter l'évènement et effectuer un traitement.

Un écouteur d'évènement peut être enregistré comme lié à un **filtre d'évènement** ou comme lié à un **gestionnaire d'évènement** suivant le comportement attendu

Les écouteurs doivent implémenter l'interface :

```
EventHandler<Event>
```

Elle est composée de l'unique méthode :

```
fun handle(event : Event)
```

qui se charge de traiter l'évènement.

Ajout d'écouteur

Pour enregistrer un écouteur d'événement sur un nœud du graphe de scène, on peut utiliser soit:

- la méthode **addEventFilter(...)** pour associer un filtre d'événement
- la méthode **addEventHandler(...)** pour associer un gestionnaire d'événement

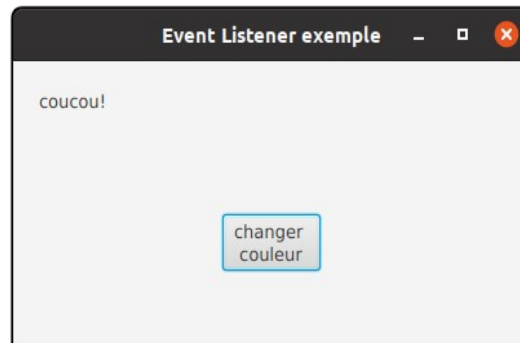
Ou pour les gestionnaires d'événement (en plus):

- Utiliser certaines méthodes spécifiques sur certains composants qui permettent d'enregistrer un écouteur d'événement en tant que propriété du composant.

Par exemple : **setOnAction(ecouteurClic)** ou **setOnKeyUp(ecouteurKey)**

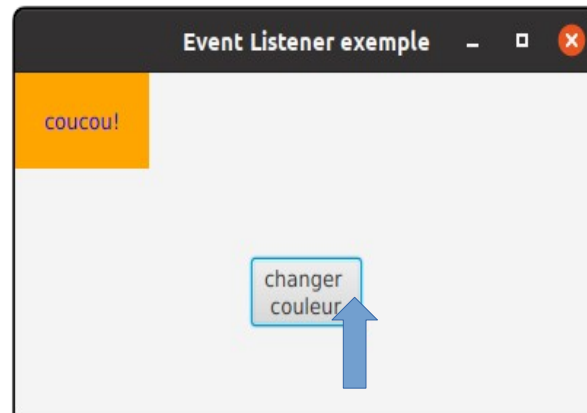
Création d'un EventListener

```
class EcouteurBouton(label : Label): EventHandler<ActionEvent> {  
    private val label : Label  
    //Constructeur  
    init {  
        this.label=label  
    }  
  
    //Code exécuté lorsque l'événement survient  
    override fun handle(event: ActionEvent ) {  
        this.label.style= "-fx-background-color: orange"  
        this.label.textFill=Color.BLUE  
    }  
}
```



Association à un composant

```
val bouton=Button("changer \n couleur")  
val label=Label("coucou!")  
val ecouteurBouton=EcouteurBouton(label)  
bouton.addEventHandler(ActionEvent.ACTION,ecouteurBouton)  
//Une autre syntaxe  
bouton.setOnAction(ecouteurBouton)
```



Maintenant au clic sur le bouton, le label se coloriera

A noter que le premier paramètre de la méthode **addEventHandler()** renseigne le type d'événement que l'on écoute.

Utilisation de lambda expression

- Les méthodes **setOnEvenement()** et **addEventHandler()** peuvent prendre respectivement comme premier et second paramètre une **lambda expression**.
- C'est une fonction anonyme => raccourci syntaxique qui permet de définir une méthode directement à l'endroit où elle est utilisée
- On ne passe pas un écouteur de type *EventHandler* en paramètre mais une fonction anonyme qui correspond au code de la fonction *handle(...)* de l'écouteur qui s'exécute lorsque l'événement ciblé survient.

```
monBouton.setOnAction{  
    label.style= "-fx-background-color: orange"  
}
```

Les lambdas expressions

Avantages :

- Diminue le nombre de classes
- Facile à implémenter avec un bon IDE
- Facilite la lecture du code

Inconvénients

- Moins de réutilisation / factorisation de code possible
- Moins de séparation des préoccupations
- **Complexifie la lecture du code et ne facilite pas la maintenance**



Privilégier l'utilisation
seulement pour des
traitements courts

Exemple d'utilisation de lambda expression

```
monBouton.setOnAction(  
    label.style= "-fx-background-color: orange"  
    label.textFill=Color.BLUE  
)
```

```
monBouton.addEventHandler(ActionEvent.ACTION,{  
    label.style= "-fx-background-color: orange"  
    label.textFill=Color.BLUE  
})
```

Les différents événements (1)

Liste des principales actions associées aux méthodes `setOnEvenement()` qui permettent d'associer des gestionnaires d'événements aux divers composants

Action de l'utilisateur	Événement	Dans classe
Pression sur une touche du clavier	<code>KeyEvent</code>	Node, Scene
Déplacement de la souris ou pression sur une de ses touches	<code>MouseEvent</code>	Node, Scene
Glisser-déposer avec la souris (<i>Drag-and-Drop</i>)	<code>MouseDragEvent</code>	Node, Scene
Glisser-déposer propre à la plateforme (geste par exemple)	<code>DragEvent</code>	Node, Scene
Composant "scrollé"	<code>ScrollEvent</code>	Node, Scene
Geste de rotation	<code>RotateEvent</code>	Node, Scene
Geste de balayage/défilement (<i>swipe</i>)	<code>SwipeEvent</code>	Node, Scene
Un composant est touché	<code>TouchEvent</code>	Node, Scene
Geste de zoom	<code>ZoomEvent</code>	Node, Scene
Activation du menu contextuel	<code>ContextMenuEvent</code>	Node, Scene

Les différents évènements (2)

Action de l'utilisateur	Évènement	Dans classe
Texte modifié (durant la saisie)	<code>InputMethodEvent</code>	<code>Node</code> , <code>Scene</code>
Bouton cliqué ComboBox ouverte ou fermée Une des options d'un menu contextuel activée Option de menu activée Pression sur <i>Enter</i> dans un champ texte	<code>ActionEvent</code>	<code>ButtonBase</code> <code>ComboBoxBase</code> <code>ContextMenu</code> <code>MenuItem</code> <code>TextField</code>
Élément (<i>Item</i>) d'une liste, ... d'une table ou ... d'un arbre a été édité	<code>ListView. EditEvent</code> <code>TableColumn. CellEditEvent</code> <code>TreeView. EditEvent</code>	<code>ListView</code> <code>TableColumn</code> <code>TreeView</code>
Erreur survenue dans le <i>media-player</i>	<code>MediaErrorEvent</code>	<code>MediaView</code>
Menu est affiché (déroulé) ou masqué (enroulé)	<code>Event</code>	<code>Menu</code>
Fenêtre <i>popup</i> masquée	<code>Event</code>	<code>PopupWindow</code>
Onglet sélectionné ou fermé	<code>Event</code>	<code>Tab</code>
Fenêtre affichée, fermée, masquée	<code>WindowEvent</code>	<code>Window</code>

Les fenêtres de dialogue

Les boîtes de dialogue sont des éléments d'une interface graphique qui se présentent généralement sous la forme d'une fenêtre affichée par une application (ou éventuellement par le système d'exploitation) dans le but :

- d'informer l'utilisateur (texte, mise en garde, ...)
- d'obtenir une information de l'utilisateur (mot de passe, choix, ...)
- ou une combinaison des deux

Une boîte de dialogue dépend d'une autre fenêtre

Boîte de dialogue modale / non modale

Modale

- L'utilisateur ne peut pas interagir avec la fenêtre dont la boîte de dialogue dépend avant de l'avoir fermée
- Une boîte de dialogue modale sera utilisée pour confirmer ou annuler une action critique (suppression de données par exemple).
 - La fenêtre principale est bloquée tant que l'utilisateur n'a pas confirmé ou infirmé son choix
 - La boîte de dialogue se ferme automatiquement dès la décision prise.

Non-modale

- L'utilisateur peut interagir avec la boîte de dialogue mais aussi avec la fenêtre dont la boîte de dialogue dépend (en laissant la boîte de dialogue ouverte).
- Une boîte de dialogue non-modale sera utilisée par exemple pour fournir à l'utilisateur une palette d'outils qu'il pourra sélectionner et appliquer sur la fenêtre principale.

La boîte de dialogue reste ouverte tant que l'utilisateur ne la ferme pas explicitement.

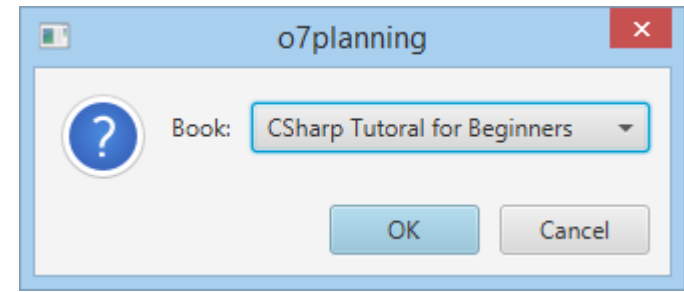
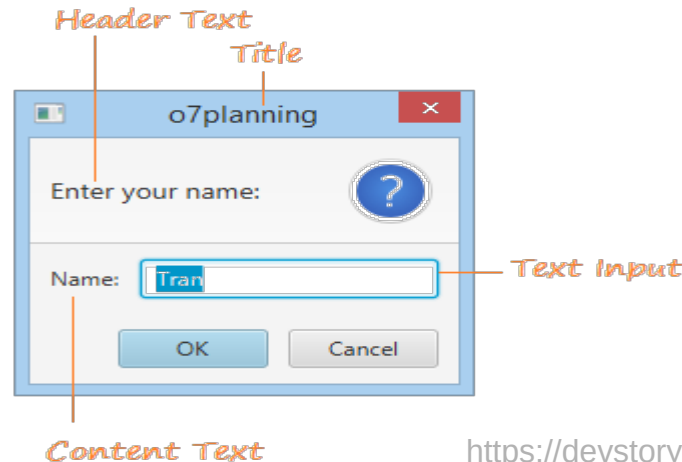
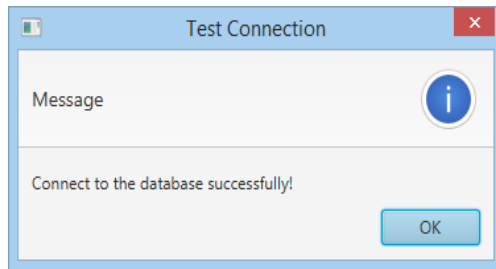
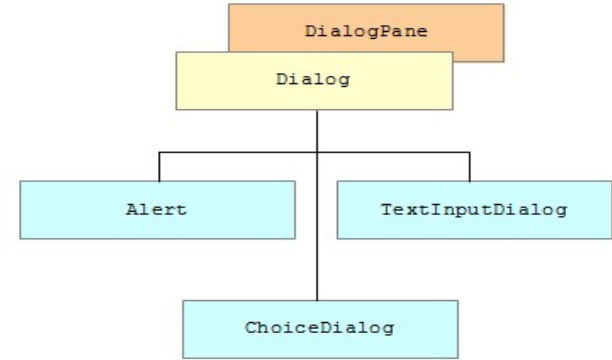
Boîte de dialogue et JavaFX

Il existe 3 types principaux de boîtes de dialogue :

Alert : permet d'afficher un message à l'utilisateur

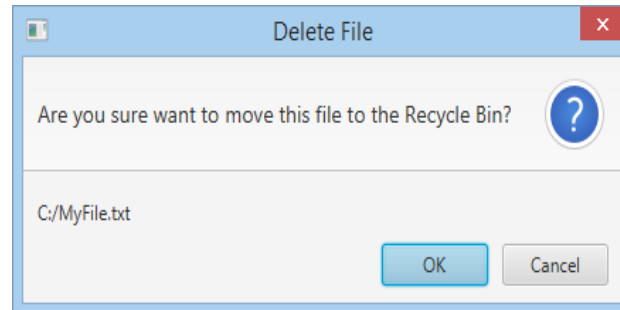
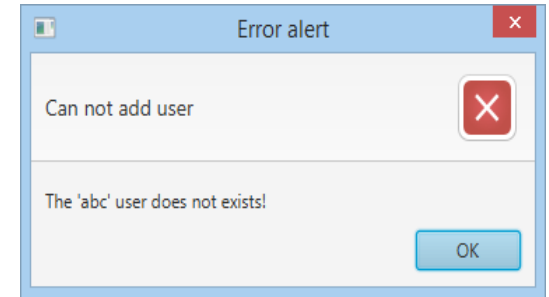
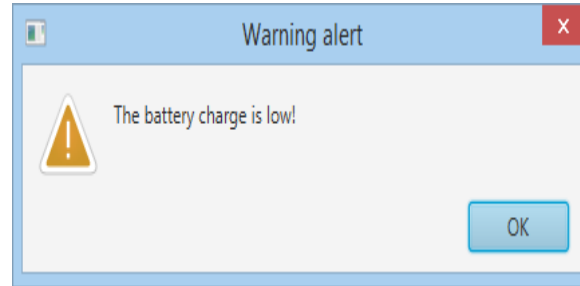
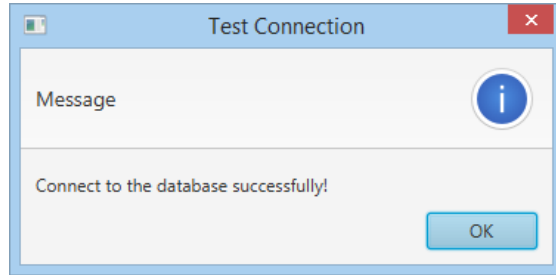
TextInputDialog : permet à l'utilisateur de saisir un texte

ChoiceDialog : permet à l'utilisateur de réaliser un choix



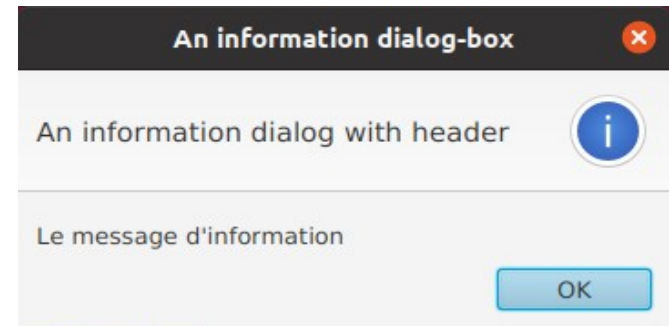
Exemple : Boîte de dialogue Alert

Plusieurs type de boîte : Information, Warning, Error, Confirmation



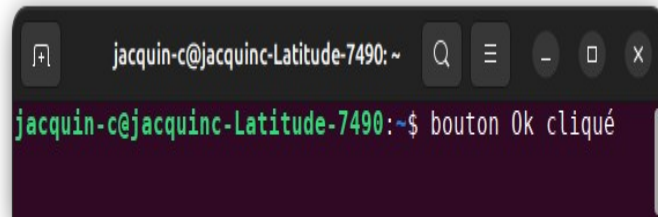
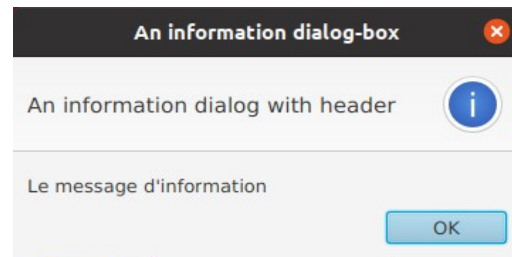
Exemple de code (1)

```
private fun showWarningAlert(){  
    val dialog = Alert(AlertType.INFORMATION)  
    dialog.title="An information dialog-box"  
    dialog.headerText="An information dialog with header"  
    dialog.contentText="Le message d'information "  
    dialog.showAndWait()  
}  
  
override fun start(premierStage Stage) {  
    ...  
    val root = BorderPane()  
    val bouton= Button("Warning Alert")  
    bouton.setOnAction{showWarningAlert()}  
    root.center=bouton  
    ...  
}
```



Exemple de code (2)

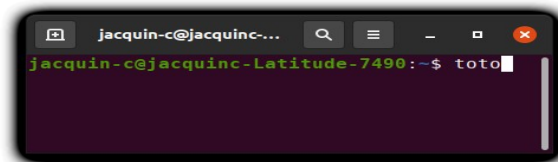
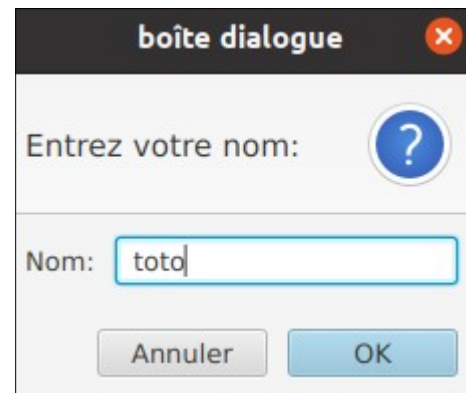
```
private fun showWarningAlert(){  
    val dialog = Alert(AlertType.INFORMATION)  
    dialog.title="An information dialog-box"  
    dialog.headerText="An information dialog with header"  
    dialog.contentText="Le message d'information "  
    result=dialog.showAndWait()  
    if ( result.get()== ButtonType.OK){  
        println("bouton Ok cliqué")  
    }  
  
    override fun start(premierStage Stage) {  
        ...  
        val root = BorderPane()  
        val bouton= Button("Warning Alert")  
        bouton.setOnAction{showWarningAlert()}  
        root.center=bouton  
        ...  
    }  
}
```



Exemple de code pour TextInputDialog

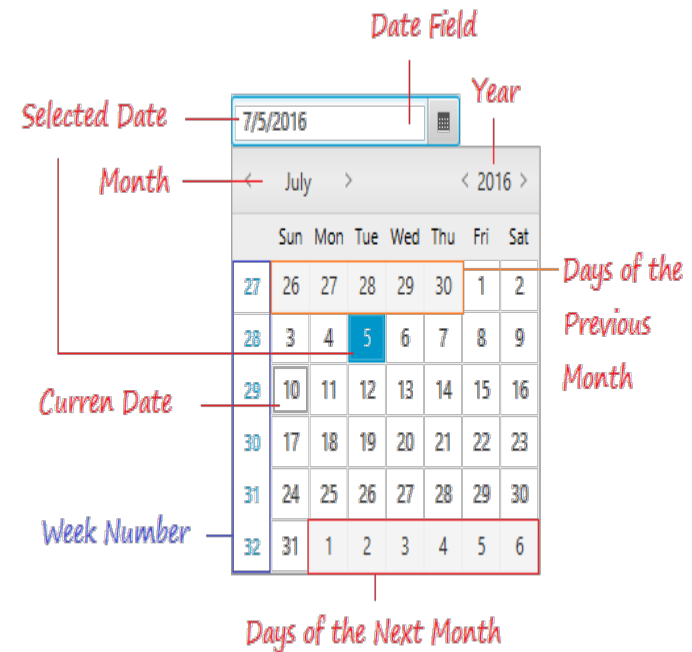
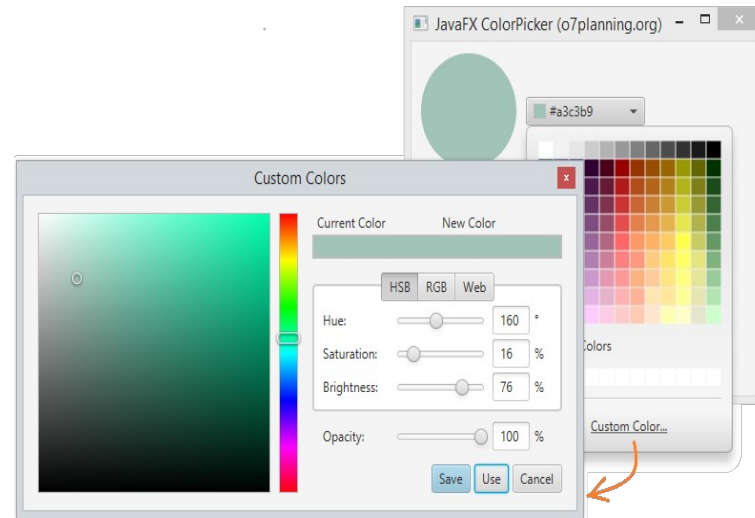
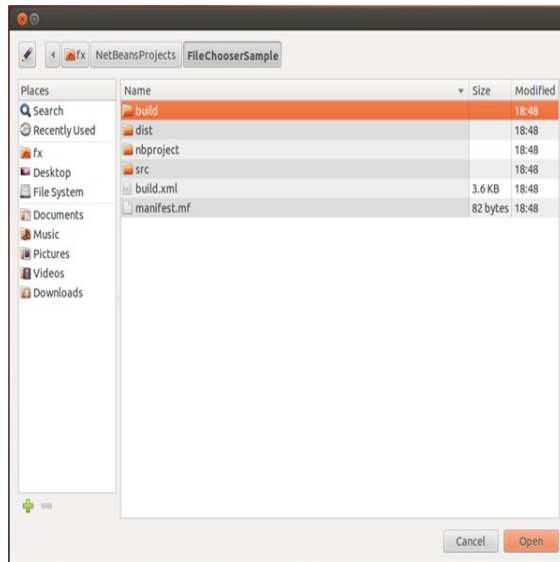
```
val dialog =TextInputDialog(" boîte dialogue")
dialog.title="boîte dialogue"
dialog.headerText="Entrez votre nom:"
dialog.contentText="Nom:"
val resultat = dialog.showAndWait()
// le traitement quand le bouton OK est cliqué
// si "annuler" alors la boîte se refermera
```

```
if (resultat.get() != "") {
    println(resultat.get())
}
```



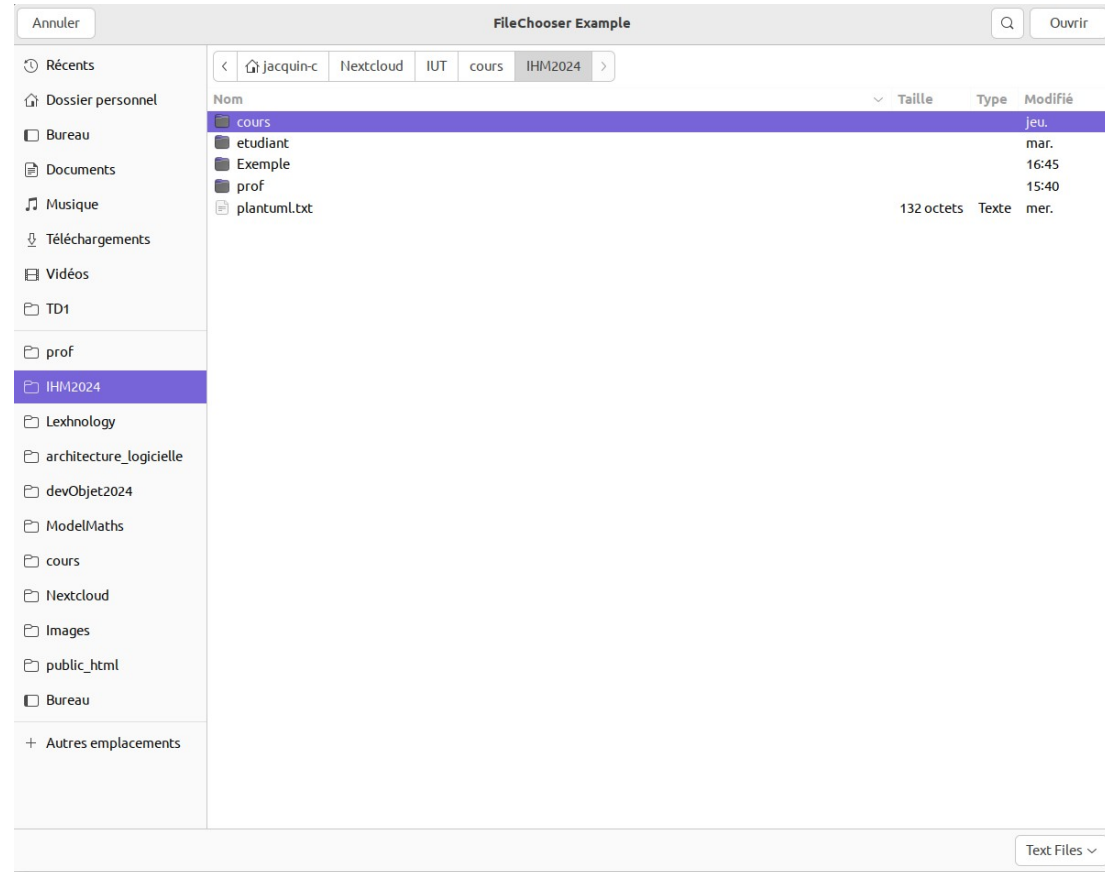
D'autres boîtes de dialogue

FileChooser, DirectoryChooser, DatePicker, ColorPicker



Exemple de FileChooser

- affiche une fenêtre qui permet de naviguer dans une arborescence de fichier et de choisir un fichier.
- la fenêtre popup s'ouvre lors de l'invocation de la méthode `showOpenDialog()` qui retourne le nom du fichier sélectionné ou une référence `null` si l'utilisateur a pressé sur *Annuler* ou a fermé la fenêtre.



FileChooser : exemple de code

```
override fun start(primaryStage: Stage) {  
    val fileChooser = FileChooser()  
    fileChooser.title = "FileChooser Example"  
    val homeDir = File(System.getProperty("user.home"))  
    fileChooser.initialDirectory = homeDir  
    fileChooser.extensionFilters.addAll(  
        FileChooser.ExtensionFilter("Text Files", "*.txt"),  
    )  
    val selectedFile = fileChooser.showOpenDialog(primaryStage)  
    if (selectedFile != null) {  
        try {  
            print(selectedFile)  
        } catch (e: Exception) {  
            System.err.println("ERROR: Unable to open the file")  
        }  
    }  
}
```